

Payment Transaction Management System

Scenario: An e-commerce enterprise handles customer transactions. The database architecture must store payment details, track payment statuses, maintain relational constraints, and generate financial reports across orders and payment transactions.

1. Create Payment Table:

```
SQL> CREATE TABLE PAYMENT(Payment_ID VARCHAR(7),Order_ID VARCHAR(7),Payment_Date DATE,Payment_Mode VARCHAR(20),Payment_Status VARCHAR(10),Transaction_Amount
NUMBER(10,2));
Table created.
```

2. Establish Relationships & Apply Constraints:

Order and Payment Relationship:

- Each order can have one payment transaction.
- Each payment belongs to one order.
- **Relationship:** Orders (1) — (1) Payment

Primary Key:

Assign Primary Key constraint to PAYMENT table on Payment_ID.

```
SQL> ALTER TABLE payment ADD CONSTRAINT pk_payment PRIMARY KEY (payment_id);
Table altered.
```

Foreign Key:

Foreign Key on PAYMENT pointing to Order_ID in ORDERS table.

```
SQL> ALTER TABLE payment ADD CONSTRAINT fk_payment_order FOREIGN KEY (order_id) REFERENCES orders(order_id);
Table altered.
```

NOT NULL:

Apply NOT NULL condition to required attributes.

```
SQL> ALTER TABLE payment MODIFY (order_id NOT NULL, payment_date NOT NULL, payment_mode NOT NULL, payment_status NOT NULL, transaction_amount NOT NULL);
Table altered.
```

Check:

- Set Check constraint to restrict Payment_Mode to allowed values ('UPI', 'Card', 'Net Banking', 'Cash').
- Set Check constraint to restrict Payment_Status to allowed values ('Success', 'Failed', 'Pending').
- Set Check constraint to ensure Transaction_Amount is strictly positive (>0).

```
SQL> ALTER TABLE payment ADD CONSTRAINT chk_payment_mode CHECK (payment_mode IN ('UPI','Card','Net Banking','Cash'));
Table altered.

SQL> ALTER TABLE payment ADD CONSTRAINT chk_payment_status CHECK (payment_status IN ('Success','Failed','Pending'));
Table altered.
```

```
SQL> ALTER TABLE payment ADD CONSTRAINT chk_transaction_amount CHECK (Transaction_amount > 0);
Table altered.
```

Insert Values into PAYMENT Table:

```

SQL> INSERT INTO payment VALUES ('PAY001','ORD001',DATE '2026-09-01','UPI','Success',520.00);
1 row created.

SQL> INSERT INTO payment VALUES ('PAY002','ORD002',DATE '2026-09-02','Card','Success',250.00);
1 row created.

SQL> INSERT INTO payment VALUES ('PAY003','ORD003',DATE '2026-09-03','Net Banking','Success',600.00);
1 row created.

SQL> INSERT INTO payment VALUES ('PAY004','ORD004',DATE '2026-09-04','UPI','Success',400.00);
1 row created.

SQL> INSERT INTO payment VALUES ('PAY005','ORD005',DATE '2026-09-05','Cash','Pending',180.00);
1 row created.

```

Display PAYMENT Table Contents:

```

SQL> SELECT * FROM PAYMENT;

```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY001	ORD001	01-SEP-26	UPI	Success	520
PAY002	ORD002	02-SEP-26	Card	Success	250
PAY003	ORD003	03-SEP-26	Net Banking	Success	600
PAY004	ORD004	04-SEP-26	UPI	Success	400
PAY005	ORD005	05-SEP-26	Cash	Pending	180

3.Manage Successful and Failed Transactions:

- **Filter Successful Transactions:**

```

SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY001	ORD001	01-SEP-26	UPI	Success	520
PAY002	ORD002	02-SEP-26	Card	Success	250
PAY003	ORD003	03-SEP-26	Net Banking	Success	600
PAY004	ORD004	04-SEP-26	UPI	Success	400
PAY006	ORD006	06-SEP-26	Card	Success	850
PAY007	ORD007	07-SEP-26	UPI	Success	420
PAY010	ORD010	10-SEP-26	UPI	Success	320

7 rows selected.

- **Filter Failed Transactions:**

```

SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Failed';

```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY009	ORD009	09-SEP-26	Net Banking	Failed	200

- **Count Payments by Status:**

```

SQL> SELECT PAYMENT_STATUS, COUNT(*) AS TOTAL_PAYMENTS FROM PAYMENT WHERE PAYMENT_STATUS IN ('Success','Failed') GROUP BY PAYMENT_STATUS;

```

PAYMENT_ST	TOTAL_PAYMENTS
Failed	1
Success	7

4. Analyze Payment Methods Used By Customers:

REPORT 1: Payment Mode Analysis

FIND:

- Number of transactions using UPI
- Number of card payments
- Most preferred payment method

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSACTIONS
-----
                4
```

```
SQL> SELECT COUNT(*) AS CARD_PAYMENT FROM PAYMENT WHERE PAYMENT_MODE = 'Card';

CARD_PAYMENT
-----
                3
```

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE      TRANSACTION_COUNT
-----
UPI                4
```

REPORT 2: Revenue Analysis

FIND:

- Total revenue generated
- Revenue by payment method
- Average transaction amount

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

TOTAL_REVENUE
-----
          3360
```

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE;

PAYMENT_MODE      TOTAL_REVENUE
-----
Net Banking        600
Card              1100
UPI               1660
```

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

AVERAGE_TRANSACTION_AMOUNT
-----
                480
```

REPORT 3: Customer Payment History

DISPLAY:

- Customer name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID,P.PAYMENT_MODE, P.TRANSACTION_AMOUNT AS AMOUNT, P.PAYMENT_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID ORDER BY P.PAYMENT_DATE;
```

CUSTOMER_NAME	ORDER_I	PAYMENT_MODE	AMOUNT	PAYMENT_STATUS
Sanjay Kumar	ORD001	UPI	520	Success
Harini Devi	ORD002	Card	250	Success
Praveen Raj	ORD003	Net Banking	600	Success
Keerthana S	ORD004	UPI	400	Success
Dinesh Kumar	ORD005	Cash	180	Pending
Pavithra M	ORD006	Card	850	Success
Ashwin Anand	ORD007	UPI	420	Success
Nithya R	ORD008	Card	360	Pending
Surya Prakash	ORD009	Net Banking	200	Failed
Lavanya M	ORD010	UPI	320	Success

10 rows selected.

Payment Analysis:

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
UPI	4	1660	415
Card	2	1100	550
Net Banking	1	600	600